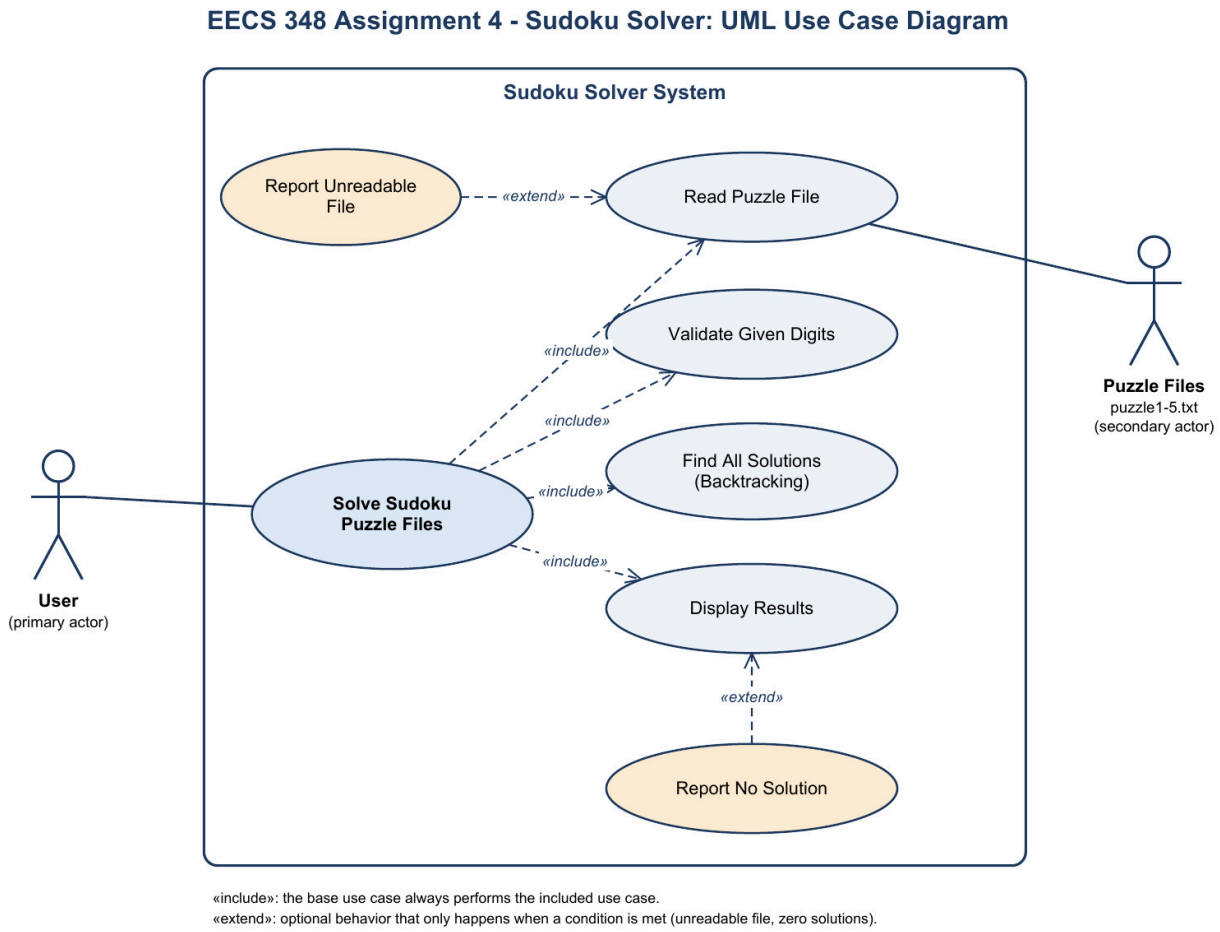


Diagram created with GenAI assistance (Claude)

A: Use Case

This program in particular has one user-visible goal: when prompted with sudoku puzzle files, show every solution to the puzzle. During this goal all the intermediary steps are just that, steps to finishing this goal, so it is modeled where solving is the primary use case with everything else as supporting use cases.

B: Brief Description

The user starts by running this Sudoku Solver. It goes through each puzzle by reading their 9x9 grids per puzzle file, it then prints the starting puzzle, uses a recursive depth-first search that has backtracking to bruteforce a solution if possible and then either prints that it found a solution or that there were "No Solutions found". If a file can't be read it gets skipped and reported to not slow down the work flow and allow for other puzzles to be completed

C: Step-by-step flow

1. The User runs the program, optionally listing puzzle file names as arguments.
2. The system builds the list of files to process (the arguments, or puzzle1.txt to puzzle5.txt if none were given).
3. The system takes the next file and prints a separator line and the puzzle file name.
4. The system reads the 81 cells from the file into a board.
5. The system prints the starting puzzle.
6. The system checks that the given digits do not already break a Sudoku rule.
7. The system finds the next empty cell and the candidate digits that can legally go in it.
8. The system tries each candidate and repeats steps 7 and 8 with one more cell filled, undoing the cell
   (backtracking) when a candidate leads to a dead end.
9. When no empty cells remain, the system saves the board as a solution and keeps searching for more.
10. When the search is exhausted, the system prints how many solutions were found and each solution.
11. The system repeats steps 3 to 10 for each remaining file.
12. The system ends the program with a status of 0

D: Detailed Description

Solve Sudoku Puzzles

The Solve Sudoku Puzzles is a use case started by the User as the primary actor and the Puzzle files as the secondary actors. The goal of the use case is for the brute force of all sudoku solutions from a given sudoku puzzle bank account which will verify their reservation booking. This use case includes Read Puzzle File, Validate Given Digits, Find All Solutions, and Display Results but is also extended by Report Unreadable File, Report No Solution. There's three conditions before this can trigger, the program needs to be compiled and work in the grading environment, the puzzles need to be in the directory of the compiled file, and each file for the program needs to hold 81 cells, in which digits range from 1-9 for the cells and underscores for the blanks. The traveler needs to choose how to pay when Make Reservation is run. The success of this use case can be done in the steps of the system showing each sudoku puzzle right next to their solutions or "No Solutions" after all of these files have been displayed and the exit code of 0. The failure of this use case shows that an unreadable file has been reported and then displays the other files that worked and then exits with a code of 1. It triggers an exception when a file cannot be opened or holds fewer than the specified cells of 81.